StamPLC PoE

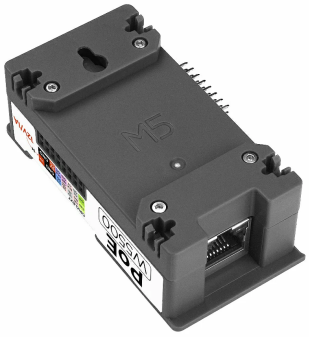
SKU:A165





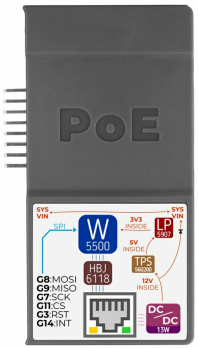






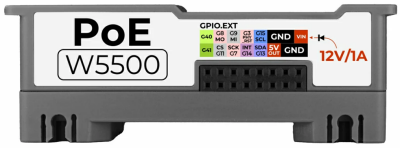


















Description

StamPLC PoE is an Ethernet control module designed for the StamPLC host. It supports PoE (Power over Ethernet) technology, enabling both data transmission and power supply through a single Ethernet cable. The module integrates a W5500 embedded Ethernet controller with a built-in TCP/IP protocol stack, providing 8 independent hardware sockets, a 10/100M Ethernet data link layer (MAC), and physical layer (PHY). It supports mainstream network communication methods such as UDP and TCP. The module connects to the network via an RJ45 Ethernet interface and communicates with the StamPLC host through an SPI interface. It is suitable for scenarios with high requirements for network stability and real-time performance, such as industrial automation production lines, centralized building control systems, energy and power distribution monitoring, and data center and communication equipment management in wired network access applications.

Features

- W5500 embedded Ethernet controller
- SPI protocol control
- Supports PoE (active Ethernet) power supply, enabling data and power over a single Ethernet cable
- Supports Ethernet Wake-on-LAN functionality via UDP
- Integrated TCP/IP protocol stack, supporting TCP / UDP / ICMP / IPv4 / ARP / IGMP / PPPoE protocols
- Supports up to 8 independent hardware sockets for concurrent communication
- Integrated 10/100M Ethernet MAC and PHY, supporting full-duplex / half-duplex operation modes
- Supports automatic network speed negotiation
- Onboard RJ45 Ethernet interface
- DIN rail mounting / mounting holes
- Development Platform
  - Arduino
  - UiFlow2

Includes

- 1 x StamPLC PoE
- 2 x Mounting connectors

Applications

- Automated production lines
- Centralized building control systems
- Energy and power distribution monitoring
- Data center and communication equipment management

Specifications

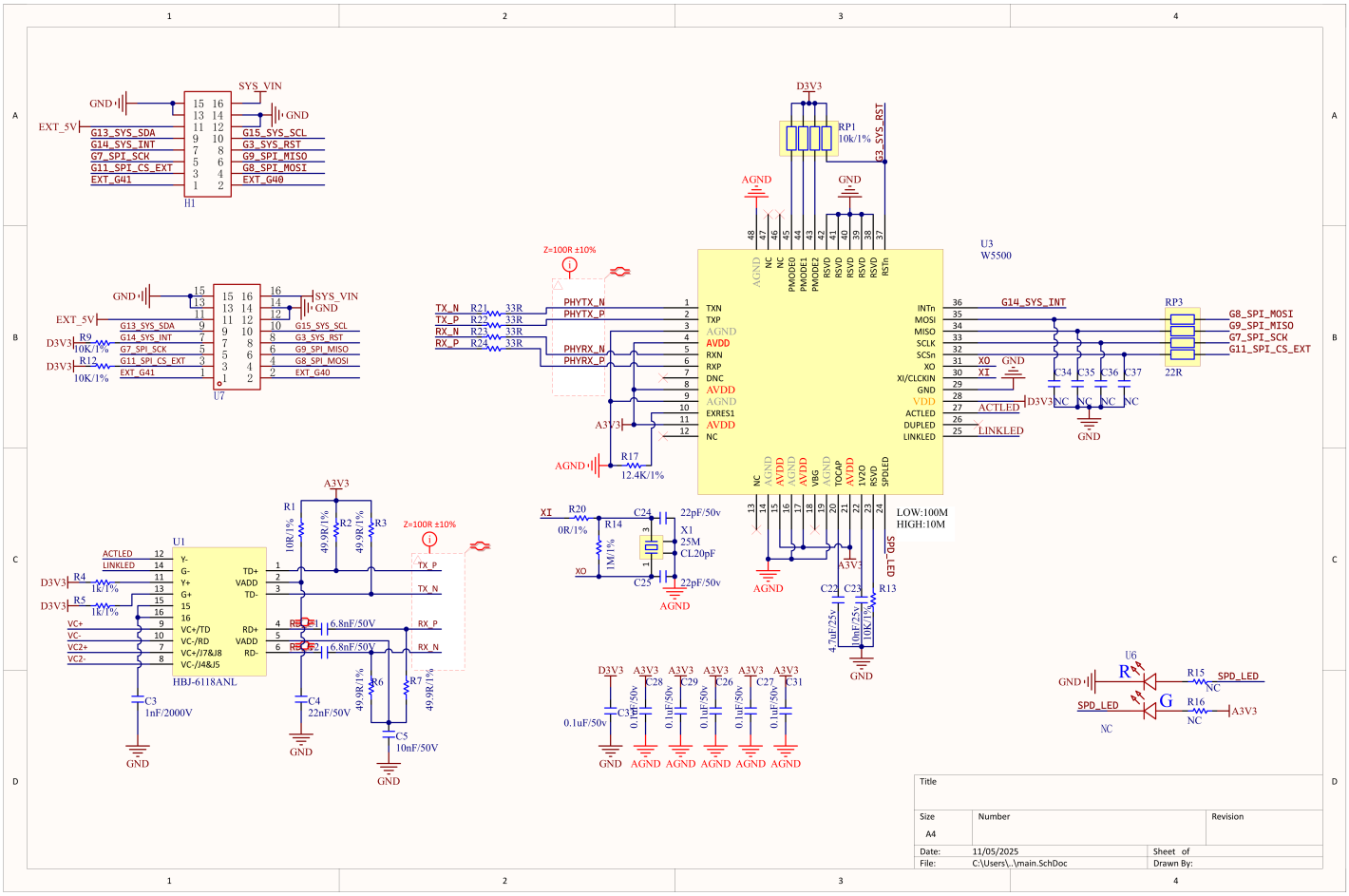
Specification	Parameter
Communication Method	SPI communication
Ethernet Controller	W5500
Ethernet Speed	10/100M adaptive
Power Supply	PoE / DC 12V / DC 5V
Hardware Sockets	8 independent hardware sockets
Supported Protocols	TCP / UDP / ICMP / IPv4 / ARP / IGMP / PPPoE
PoE Features	Complies with IEEE802.3af, supports DC 37 ~ 57V input
PoE Power	Maximum 13W
Interface Type	2 × 8P 2.54mm male / female headers, RJ45 Ethernet interface
MAC/PHY	Integrated 10/100M MAC & PHY
Installation Method	DIN rail mounting / mounting holes
Product Size	77.0 x 38.0 x 27.1mm
Product Weight	46.0g
Package Size	80.0 x 47.0 x 30.0mm
Gross Weight	52.0g

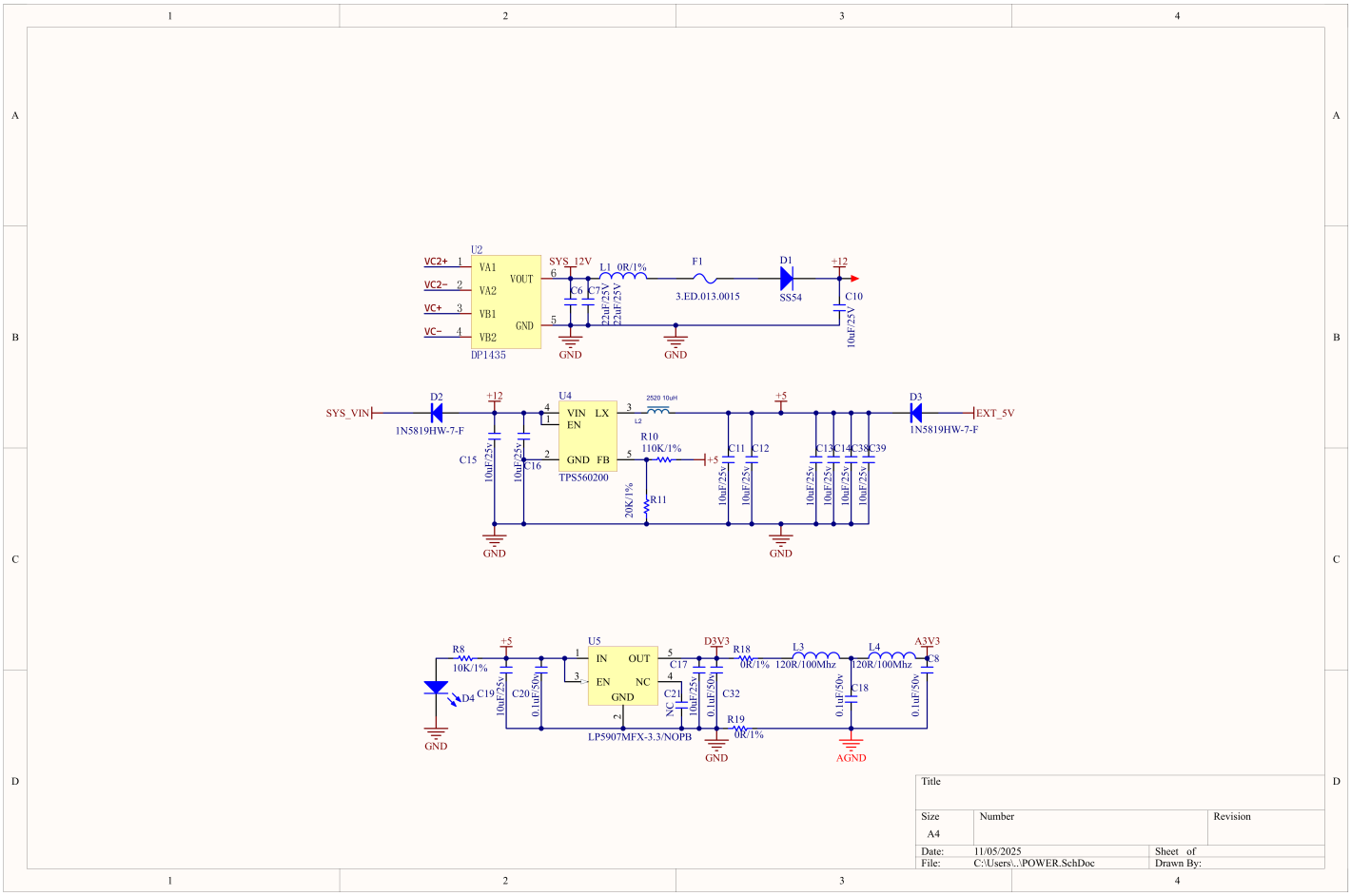
StamPLC and StamPLC PoE Connection Diagram



Schematics

- StamPLC PoE Schematics PDF





PinMap

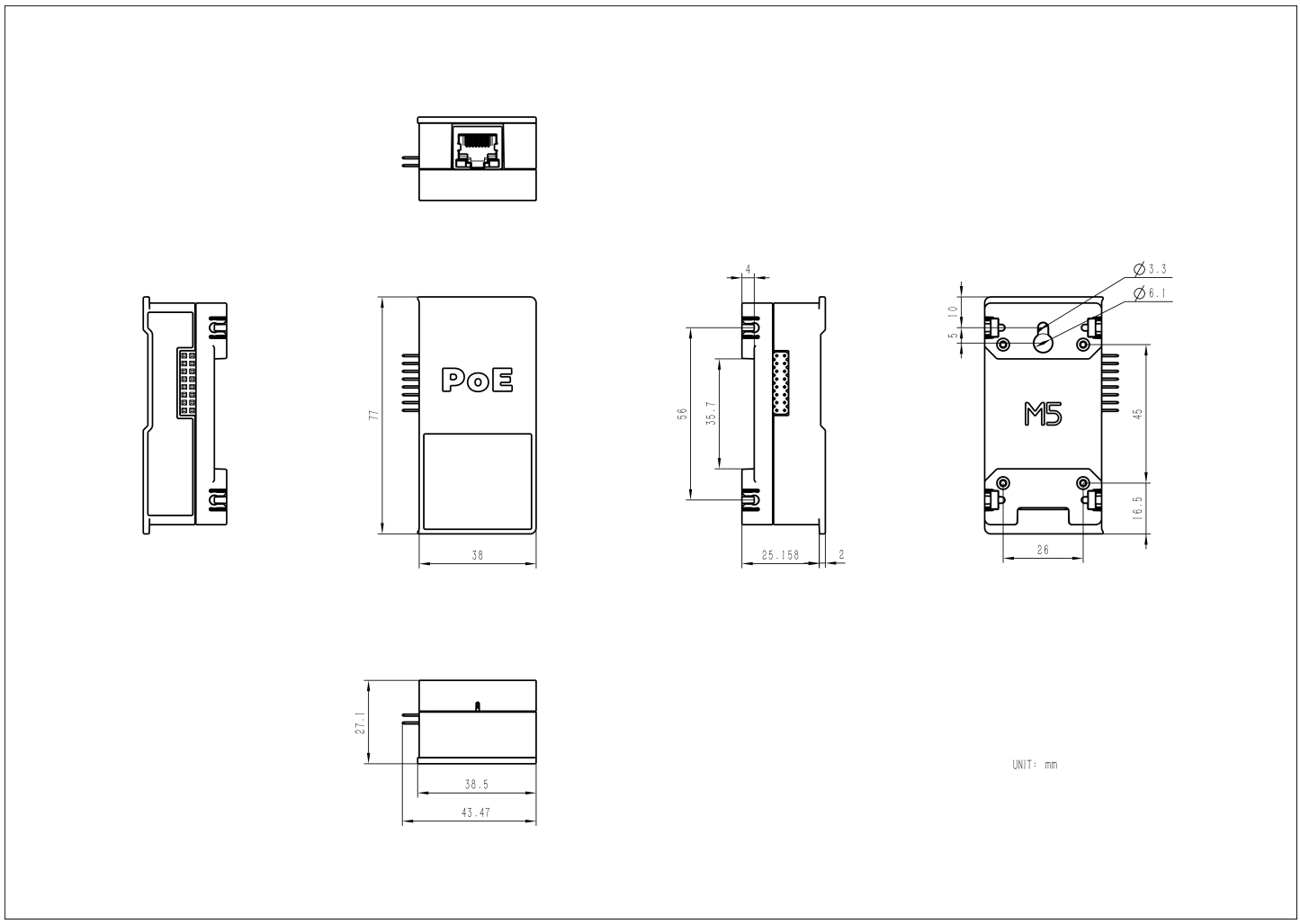
StamPLC	G8	G9	G7	G11	G3	G14
StamPLC PoE	MOSI	MISO	SCK	CS	RST	INT

StamPLC-Bus

PIN	LEFT	RIGHT	PIN
VIN	1	2	GND
GND	3	4	GND
GND	5	6	EXT_5V
SCL	7	8	SDA
RST	9	10	INT
SPI_MISO	11	12	SPI_SCK
SPI_MOSI	13	14	SPI_CS
	15	16	

Model Size

- StamPLC PoE Model Size PDF



Datasheets

- W5500

Softwares

Arduino

- StamPLC PoE Arduino Quick Start

UiFlow2

- StamPLC PoE UiFlow2 Docs

ESP-IDF

- W5500 ESP-IDF Component

Software Notes

When using StamPLC together with the StamPLC PoE expansion module, please note the following:

1. The RST of the `StamPLC LCD` screen, the RST of the internal IO expander chip (`PI4IOE`), and the RST of the external `StamPLC PoE` expansion module are interconnected.
2. Some driver libraries control the RST pin when initializing the StamPLC PoE Ethernet module (W5500), causing a pin reset operation. This will also reset the already initialized screen. It is recommended to configure the RST pin as none (`-1`) during Ethernet driver

initialization.

3. The INT pin of the internal IO expander chip (`PI4IOE`) is connected to the interrupt signal of the external StamPLC PoE expansion module. When the expansion module generates an interrupt signal, it may cause abnormal occupation of the interrupt bus of the IO expander chip (`PI4IOE`). To avoid this condition, configure the IO expander chip as follows:

Set the interrupt mask register of `PI4IOE` to fully disable interrupts.

```
#define PI4IO_REG_INT_MASK 0x11
#define PI4IO_REG_IRQ_STA  0x13
#define PI4IO_REG_CHIP_ID  0x01

static void prepare_shared_int_gpio(void)
{
#if CONFIG_ETHERNET_SPI_INT0_GPIO >= 0 && CONFIG_ETHERNET_SPI_POLLING0_MS == 0
    i2c_bus_device_handle_t pi4io = io_expander.get_device_handle();
    if (pi4io) {
        i2c_bus_write_byte(pi4io, PI4IO_REG_INT_MASK, 0xFF);
        uint8_t dummy = 0;
        i2c_bus_read_byte(pi4io, PI4IO_REG_IRQ_STA, &dummy);
        i2c_bus_read_byte(pi4io, PI4IO_REG_CHIP_ID, &dummy);
    }
    i2c_bus_handle_t bus = io_expander.get_bus_handle();
#endif
}
```

Below is an example project for handling the above conflict issue in the ESP-IDF environment:

- ESP-IDF StamPLC PoE Demo

Video

- StamPLC PoE Product Introduction and Feature Demonstration

A165_StamPLC_PoE_video_EN.mp4